A MINDMAP STUDIO GUIDE

Thinking in Maps

A practical guide to mind mapping — the technique and the canvas — with MindMap Studio.

Dann Bleeker Pedersen

Generated 2026-06-14
mindmap-studio.struktureretsundfornuft.dk

Contents

Foreword
Your first map
The anatomy of a map
Structuring ideas
Enriching nodes
Navigating large maps
Sharing and exporting
Presenting and workshops
Appendix A -- Keyboard reference
Appendix B -- Format reference
Appendix C -- Further reading

Foreword

Why a book about drawing bubbles and lines?

A mind map looks like the simplest thing in the world: a word in the middle, some branches, a few more words. People sketch them on napkins. So why a whole book?

Because the simplicity is the point, and the discipline behind it is easy to miss. A mind map is a thinking tool disguised as a doodle. It externalises the shape of an idea -- what depends on what, what sits beside what, what is still missing -- so your working memory stops juggling and your eyes can do the work instead. Done well, it turns a vague "I should plan this" into a structure you can argue with.

This book is in two voices at once. The first is about **the technique** : how to grow a map that actually helps you think, when to branch, when to stop, how to keep a big map from collapsing into noise. The second is about **the tool** : MindMap Studio , a small, fast, local-first app that runs in your browser, keeps your maps on your own machine, and asks for no account and no network. Every technique in the book is shown end-to-end in the app, with the exact keys and controls you press.

Who this is for

You don't need any prior experience with mind mapping, and you don't need to be a designer or a "visual thinker" -- a phrase that does more harm than good. If you can write a bulleted list, you can build a mind map, and this book will take you from a single node to a map you'd happily present.

If you're coming from MindManager, XMind, or a wall of sticky notes, you'll find the moves familiar and the friction lower. MindMap Studio is deliberately small: it does brainstorming and structuring well, and leaves project-management gravity to other tools.

How to read it

- **Part 1 -- Getting started** gets a real map on your screen and explains the few ideas the whole app rests on.
- **Part 2 -- Building maps** is the craft: structure, enrichment, and keeping large maps navigable.
- **Part 3 -- Sharing your thinking** covers getting the map *out* -- exporting, presenting, and running a session with other people.
- The **appendices** are reference: a keyboard card, the file-format details, and where to read more.

Read Part 1 in order. After that, jump to whatever you need. The map you build in Chapter 1 carries through the book, so it's worth following along in the app as you go.

Everything autosaves. You can't lose your place. Let's begin.

Your first map

From a blank canvas to a real structure in five minutes

Open MindMap Studio . The first time, you land on a sample map -- *Q3 Retail Plan* -- showing off notes, markers, a relationship arrow and a boundary. Click around it to get a feel for the canvas, then start your own: press **+ New...** in the toolbar and pick **Blank** .

You now have a single node that says *Untitled map* . That node is the **root** -- the one idea everything else hangs off. Double-click it (or just start typing) and give it a real subject. A map about planning a conference might have a root that simply says *Conference* .

The three keys that do everything

Almost all of map-building is three keys. Select the root, then:

- **Enter** adds a *sibling* -- another node at the same level. From the root, the first Enter gives you your first branch.
- **Tab** adds a *child* -- a node one level deeper, attached to the node you have selected. This is how a branch grows outward.
- **Delete** removes the selected node (and everything under it).

That's the whole loop: Tab to go deeper, Enter to stay at the same level, type to label, Delete to prune. Try it. Select *Conference* , press **Tab** , type `Venue` . Press **Enter** , type `Programme` . Press **Enter** , type `Budget` . You have three branches. Select `Venue` , press **Tab** , and add `Shortlist` , `Site visit` , `Contract` as its children. Within a minute you have a small tree.

> A fourth key is worth learning early: **Ctrl+Enter** inserts a line break *inside* a node, for when a label genuinely needs two lines. Use it sparingly -- short labels keep a map readable.

Moving around

Click any node to select it; the arrow keys walk you between parent, child and siblings. Drag the canvas background to pan; scroll or pinch to zoom. If you ever lose the map off the edge of the screen, the **Fit** button in the toolbar frames the whole thing again.

You can't lose it

MindMap Studio saves continuously to your browser's local storage (IndexedDB) as you type. Close the tab, reopen it tomorrow, and the map you were working on is exactly where you left it. There is no Save button because there is nothing to save -- it has already happened. There is also no server: the map never leaves your machine unless you explicitly export it, which is Chapter 6.

Undo is your safety net

Made a mess? **Ctrl+Z** undoes; **Ctrl+Y** (or **Ctrl+Shift+Z**) redoes. Undo is model-aware -- it reverses the actual change to the map, not just a visual tweak -- so you can experiment with structure freely and walk it back if a branch turns out wrong.

Now you try

Start a **Blank** map and build something real to you in five minutes -- a trip, a project, a decision you're weighing. Name the root, then grow three or four branches with a couple of children each, using only **Tab** (deeper), **Enter** (same level), and typing. Delete a branch and **Ctrl+Z** it back, just to feel the safety net. Then close the tab and reopen it: the map is exactly where you left it. That loop -- Tab, Enter, type, undo, and it already saved itself -- is the whole foundation the rest of this book builds on.

That's enough to build maps. The next chapter slows down for a moment to explain *what* you've been building -- the handful of ideas the rest of the app rests on.

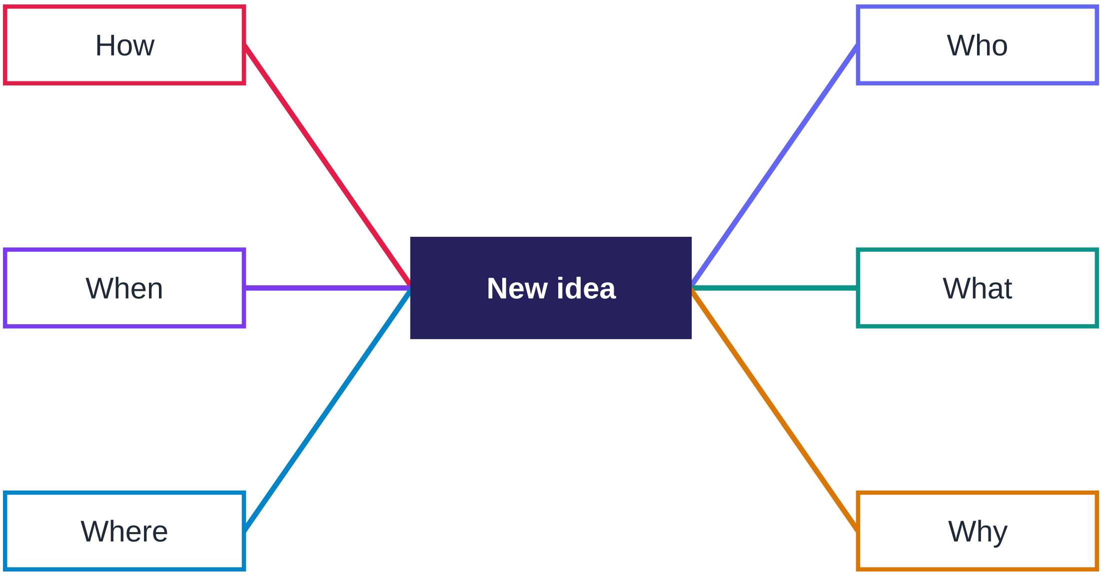
The anatomy of a map

A handful of ideas the whole app rests on

You can use MindMap Studio without ever reading this chapter. But ten minutes here will make every later feature feel obvious instead of arbitrary, because they all act on the same small set of parts.

The shape: one centre, many branches

A mind map is a tree with a single root in the middle and branches radiating outward. The classic starting point is a central idea surrounded by the six questions every plan eventually has to answer:



A mind map: one central idea, six branches — the Brainstorm template.

This is exactly what the built-in **Brainstorm** template gives you (you'll meet the other templates in Chapter 7). Each branch can grow its own children, and theirs can grow more, as deep as the thought goes. The radial shape isn't decoration: putting the subject in the centre keeps every branch equidistant from it, so no single line of thinking dominates just because it happens to be at the top of a list.

The node: the only real object

Everything on the canvas is a **node** . A node has:

- a **topic** -- the text you see;
- an optional set of **markers** -- small icons or tags (Chapter 4);
- an optional **note** -- longer prose attached behind the node (Chapter 4);
- an optional **image** ;

- and **children** -- the nodes hanging off it.

The root is just a node that happens to have no parent. A leaf is just a node with no children. There is no special "task" type or "milestone" type -- MindMap Studio keeps the model deliberately small. That smallness is why import, export and undo all behave predictably: there are fewer kinds of thing to get wrong.

Two things that aren't nodes

Two features draw on the canvas without being part of the tree:

- A **boundary** is a soft, shaded box drawn around a node and everything beneath it, grouping a branch visually ("everything in here is Phase 1").
- A **relationship** is an arrow drawn from one node to another, expressing a link that the tree structure can't -- a dependency, an influence, a "see also" across branches.

Both are covered in Chapter 3. The point for now: the tree carries the *hierarchy* , and these two carry the *exceptions* to it.

The canonical model

Under the surface, your map is a plain data structure -- a root node, its children, each node's topic and notes and markers. MindMap Studio treats that structure as the single source of truth. The colourful canvas is one *view* of it; the outline panel (Chapter 5) is another; an exported Markdown file is a third. Edit the map on the canvas and the underlying model updates; every other view follows.

This is worth internalising because it explains the app's most reassuring property: **what you see is always backed by real data you can take with you.** A map is never trapped in a proprietary blob. Chapter 6 shows you how to round-trip it through Markdown, JSON, OPML and more -- losslessly, in the case of JSON.

Now you try

Open the sample *Q3 Retail Plan* map and name the parts out loud: which node is the **root** , which are **branches** , where's the **boundary** , where's the **relationship** arrow. Then prove the "one model, many views" idea to yourself -- open the **Outline** panel (Chapter 5) and watch the same tree appear as an indented list, or export to **Markdown** (Chapter 6) and read your map as plain text. Same data, three faces. Now **rename a node on the canvas** and look again: the outline and the Markdown already show your edit, because the canvas writes straight to the one model every view reads from -- nothing to "save", nothing to sync.

With the parts named, the rest of the book is just learning what you can do to them.

Structuring ideas

Re-parenting, grouping, and the links a tree can't hold

A first draft of a map is rarely in the right shape. A branch you started under *Venue* turns out to belong under *Budget* ; two ideas you thought were separate turn out to be the same. Restructuring is not a failure -- it *is* the thinking. This chapter is about moving things around without fear.

Drag to re-parent

To move a node -- and everything beneath it -- to a new home, just **drag it onto its new parent** . Drop `Catering` from under *Venue* onto *Budget* and the whole sub-branch travels with it, re-coloured to match its new branch. There is no cut-and-paste dance; the drag *is* the move. Because the underlying model updates atomically, **Ctrl+Z** puts it back if you misjudged the drop.

A good habit: build fast and loose first -- get every idea onto the canvas as a node, wherever -- then spend a second pass dragging things into the structure that emerges. Trying to get the hierarchy right while you're still generating ideas slows both down.

Layout direction

By default MindMap Studio balances branches on both sides of the root. For some maps a one-sided layout reads better -- an agenda or a process that flows top-to-bottom in your head often wants everything going **right** . The layout control in the toolbar switches between **both** , **right** , and **left** . It's a view setting; it changes nothing about the structure, so flip it freely to see which framing helps.

Boundaries: grouping a branch

Sometimes a branch needs to be visibly *a thing* : "everything in here is out of scope", "this cluster is Phase 1". Select the node at the top of the branch and click **Group** : a shaded, rounded **boundary** box is drawn around it and all its descendants, and double-clicking the box's label chip lets you name it. The boundary follows the branch as you edit it, so it keeps enclosing the right nodes even after you add or move children.

Boundaries are the right tool when the grouping is *hierarchical* -- it lines up with a single branch. When the grouping cuts across the tree, you want the next feature instead.

Relationships: the arrows across the tree

The tree is good at "X is part of Y". It is silent about "X depends on Y" when X and Y live on different branches. That's what **relationships** are for: draw an arrow from one node to another, anywhere on the canvas, and you've recorded a link the hierarchy couldn't express. A risk on the *Budget* branch that threatens a deliverable on the *Programme* branch; a decision that unblocks three others; a "see also". Use them sparingly -- a map laced with dozens of arrows is as hard to read as no structure at all -- but a handful of well-placed relationships often carry the most important information on the page.

> **Keep the tree honest.** Before drawing a relationship, ask whether the two nodes are really on different branches, or whether one should simply be re-parented under the other. An arrow is the right answer for genuine cross-links; it's the wrong answer for a hierarchy you just haven't tidied yet.

Floating topics

Not everything has to connect to the root. A **floating topic** is a node (or small sub-tree) that sits on the canvas unattached -- a parking lot for ideas you're not ready to place, a caption, a note-to-self. MindMap Studio renders floating topics imported from other tools in a labelled "Floating topics" branch -- and they're fully editable, so they double as a staging area you can rename, grow, and prune while you decide where something belongs. Drag one onto a branch and it joins the tree; drag a branch topic out and it floats free.

Now you try

Open your conference map (or any map with a few branches). Pick a leaf that sits under the wrong parent and **drag it** where it belongs -- watch the whole sub-branch travel and re-colour. Then choose one branch that's genuinely "a thing" and give it a **boundary** . Finally, find two nodes on *different* branches that depend on each other and draw a **relationship** arrow between them. Step back: the tree now carries the hierarchy, the boundary carries the grouping, and the arrow carries the one link the tree couldn't. If you can't find a real cross-branch link, that's a good sign -- don't invent one.

Two more moves worth a try. Flip the **layout direction** to right-only and back -- same structure, a different shape on the page; keep whichever frames the map better. Then drag a branch topic *out* into open space to make it a **floating topic** -- a parking lot for an idea you're not ready to place -- and drag it back onto a branch to re-attach it. Structure isn't a cage; it's something you reshape as the thinking firms up.

With structure under control, the next chapter makes individual nodes carry more.

Enriching nodes

Notes, markers, images, and making a map look like it means it

A bare tree of labels is often enough. But a node can carry far more than its topic, and used with restraint, that extra detail turns a sketch into a document you can hand to someone else.

Notes: the prose behind the bubble

A topic should stay short. The paragraph it deserves goes in a **note** . Select a node, open the **Notes** panel, and write. Notes accept **Markdown** -- headings, bold and italic, bullet lists, inline `code` , and links -- and the panel shows a live preview, so the formatting you type is the formatting you get.

Notes are where a map stops being a brainstorm and becomes a brief. The node says *Vendor decision* ; the note records the three options, the criteria, and why you chose the one you did. The canvas stays scannable; the detail is one click away on whichever node owns it.

Markers: status at a glance

A **marker** is a small icon or tag pinned to a node. Open the **Markers** palette and click to toggle one on the selected node: a priority flag, a tick, a question mark, a face. Markers are how a map carries *state* without words -- a row of green ticks and one red flag tells a reviewer where to look before they've read a single label.

Pick a small vocabulary and stick to it. "Red flag means blocked, tick means done, question mark means undecided" is a convention a whole team can read at a glance; fifteen different icons used once each is just clutter.

> Markers imported from a MindManager `.mmap` file are mapped to the closest emoji, so a map you bring in from elsewhere keeps its visual cues rather than arriving as bare text.

Images

A node can hold an **image** -- a screenshot, a logo, a photographed whiteboard, a chart. Attach one to the selected node and it renders inline on the canvas. An image is worth a paragraph of note when the thing you're describing is itself visual.

Styling: shape, fill, border, bold

Beyond markers, individual nodes can be **styled** : change a node's shape, fill colour, border, or weight; bump a topic's font size or colour. Styling earns its keep when it *encodes* something -- the three "decision" nodes all share a fill, the headline branch is bolder than the rest -- and costs you readability when it's merely decorative. Style to create a pattern the reader can rely on, not to make the map "pop".

Themes: the whole canvas at once

Where per-node styling is a scalpel, a **theme** is a coat of paint for the entire map. The theme gallery offers a few presets -- **Light** , **Dark** , **Ocean** , **Sunset** -- each a coordinated palette for the background, branches and text. Dark themes read well on a projector in a dim room; light themes print cleanly. Switching theme never touches your content, so try a few and keep whichever helps you see the map.

A note on restraint

Every feature in this chapter can be overused. The test is always the same: *does this make the map easier to think with?* A note that captures a decision -- yes. A marker convention a team shares -- yes. Six fonts and a gradient on every node -- no. The most useful maps are usually the plainest ones with enrichment applied exactly where it carries meaning.

Now you try

Take a node that's carrying too much in its label and move the detail into a **note** (open the Notes panel and write a sentence or two in Markdown). Shorten the topic to a handful of words. Then agree a tiny **marker** convention with yourself -- say, a flag for "blocked" and a tick for "done" -- and apply it to three or four nodes. Read the map again: the canvas should be more scannable than before, with the depth one click away. If you reach for a fifth marker colour, stop -- the small vocabulary is the point.

Now add a layer of meaning. Attach an **image** to one node -- a screenshot or a logo -- and watch it render inline. Pick three related nodes (say, the ones that represent decisions) and give them a shared **fill** from the Style bar so the pattern reads at a glance; bump your headline branch's **font** up a size so the eye starts there. Finally, open the **theme** gallery and switch between Light and Dark -- your content doesn't change, only its coat of paint, so keep whichever helps you see the map. The rule throughout: styling that *encodes* something earns its place; styling that merely decorates doesn't.

The next chapter is about the opposite problem: when a map gets big, how do you keep finding your way around it?

Navigating large maps

Outline, find, collapse, and links between maps

A map with thirty nodes fits on a screen. A map with three hundred does not, and the features that felt optional at small scale become the difference between a map you use and a map you abandon. This chapter is about staying oriented.

Collapse and expand

Every branch can be **collapsed** to hide its children behind its parent, and expanded again to reveal them. Collapsing is how you control altitude: fold everything down to the top two levels to see the shape of the whole plan, then open just the branch you're working in. The toolbar's **collapse all** and **expand all** controls do it to the whole map at once -- collapse-all then open one branch is the fastest way to present a large map without overwhelming the room.

Fit

Lost? The **Fit** button reframes the entire map to the viewport in one click. It's the "take me home" of the canvas, and worth wiring into muscle memory: zoom in to work, hit Fit to see where you are.

The minimap and zoom

In the bottom-right corner sits a **minimap** -- a shrunk-down overview of the whole map, with a highlighted rectangle showing the slice you're currently looking at. On a big map it answers the question "where am I, and what else is out there?" at a glance. Click or drag inside it to jump the main view somewhere else: the rectangle follows your pointer and the canvas pans to match, so the minimap doubles as a fast way to travel across a map too large to scroll comfortably.

Below it are the **zoom controls** -- minus, a live percentage, plus, and a fit button. They do the same job as the mouse wheel but give you a precise readout and a deliberate step, which matters when you're lining a map up for a screenshot or a screen-share. The percentage tells you exactly how far in you are; the fit button is the same "take me home" as **Fit** above, kept within thumb's reach of the zoom buttons.

The outline panel

The **Outline** panel shows your map as an indented list -- the same tree, read top to bottom instead of radiating outward. It's invaluable for three things: seeing the whole structure linearly, jumping straight to a node (click it in the outline and the canvas focuses it), and spotting structural problems a radial layout can hide, like a branch that's gone six levels deep while its siblings stayed shallow.

The outline has its own **filter** box: type a few letters and it narrows to matching topics, so a long map becomes a short list you can scan.

Find and replace

Press **/** anywhere to jump straight to **Find** . It searches both topics *and* notes, so a term you only mentioned in a note is still findable. Matches are highlighted and you can step between them. **Replace** turns find into a tidy-up tool: rename a project that got called three slightly different things, fix a term you decided to change, all in one pass.

> The `/` -to-find shortcut is the single most useful key on a big map. When you can't remember where something is, don't hunt -- press `/` and type.

Links between maps

Some subjects are too big for one map. Rather than one sprawling canvas, MindMap Studio lets you keep a **library** of maps (Chapter 7) and draw **cross-map links** between them: a node in your *Strategy* map can link to your *Q3 Plan* map, so clicking it opens the other map. This is how you build a small atlas instead of one unreadable continent -- a high-level map whose nodes are doorways into the detailed maps beneath them.

Search every map

Find searches the map you're in. Once your library grows into an atlas, **All maps** searches *all* of them at once -- every topic and note, floating topics included -- and picking a result opens that map and lands on the node. It's the companion to cross-map links: links are the doorways you placed on purpose, library search is for when you can't remember which map a thing is in.

A working rhythm for big maps

Put together, the loop looks like this: **collapse all** to see the shape, open the branch you need, **/** to jump to a specific node, work, **Fit** to re-orient, repeat. The outline panel stays open on the side as a table of contents. None of these moves is clever on its own; the habit of using them together is what keeps a three-hundred-node map feeling as manageable as a thirty-node one.

Your workspace, remembered

The panels you work with -- the **Outline** on the side, the **Notes** editor -- stay how you left them. Close the app with the outline open and it's open when you come back; the app remembers your panel layout between sessions, so you don't rebuild your workspace every morning. It's a small thing, but it's the difference between a tool that settles into your habits and one you have to re-arrange every time you sit down.

Now you try

Find (or build) a map big enough that it doesn't fit on screen. **Collapse all** , then open just one branch and talk yourself through it. Press **/** and jump to a node you only mentioned in a *note* -- prove to yourself that Find searches notes, not just topics. Open the **Outline** panel and use its filter to narrow a long map to a short list. Zoom right in on one node, then press **Fit** -- watch the whole map snap back into view; that's your "take me home". If you keep more than one map, open **All maps** and search for a term you know lives in a *different* one -- watch it open that map and land on the node. While you have two maps, add a **cross-map link** : point a node in one at the other, then click it and watch the app hop maps -- that's how a high-level map becomes a set of doorways into the detailed ones beneath it. Then **reload the page** : the Outline panel is exactly where you left it, because the app remembered your workspace. The goal isn't to memorise the controls; it's to feel how much calmer a big map gets when you drive it at the right altitude instead of staring at the whole thing at once.

Part 3 turns outward: getting the map off your screen and in front of other people.

Sharing and exporting

Getting the map out -- losslessly when it matters

A map you can't get out of the app is a map held hostage. MindMap Studio is built on the opposite principle: your work is plain data, and there are many doors out. This chapter is the catalogue of them, and when to use which.

The export menu

The toolbar's **Export** menu offers, in rough order of fidelity:

- **JSON** -- the native format, and the only **lossless** one. Topics, notes, markers, images, boundaries, relationships, styling: everything in the model survives a round trip. Export to JSON for backup, for moving a map between machines, or any time you might want to re-open it later with nothing lost.
- **Markdown** -- the map as an indented outline (`#` root, nested bullets). Perfect for pasting into a document, a wiki, or a pull request. It's round-trippable as structure, though it naturally drops canvas-only detail like colours and arrows.
- **OPML** -- the interchange format outliners speak. Use it to hand your structure to another outlining tool.
- **FreeMind / Freeplane (`.mm`)** -- the most widely-read mind-map format. Carries the topic tree, links, folded branches, and notes, so a map opens in FreeMind, Freeplane, XMind, and the many tools that import `.mm` . The bridge to almost any other mind-mapper.
- **Mermaid (`.mmd`)** -- the `mindmap` text format you embed in Markdown, a README, or a wiki that renders Mermaid (GitHub, GitLab, Notion, many docs tools). For when the map should live *as text* inside something you're already writing.
- **PNG** and **SVG** -- the map as a picture. PNG for slides and chat; SVG when you want a crisp, scalable image that survives zooming.
- **HTML** -- a self-contained web page of the map, openable in any browser with nothing installed.
- **HTML slide deck** -- the walk-through (Chapter 7) as a standalone, navigable presentation in a single file: an overview slide, then one per branch, advanced with the arrow keys or a click. Hand someone the file and they can present your map with nothing installed.
- **PowerPoint (`.pptx`)** -- a real, editable slide deck: an overview slide, then one per branch with its points as bullets. For when the deck has to live in PowerPoint -- a house template to apply, or a room to run from the corporate machine.
- **Word (`.docx`)** -- the map as an editable outline document: a title, indented bulleted topics, and notes as italic lines. For when the next step lives in a word processor -- minutes, a brief, a hand-off to someone who doesn't use the app.
- **Excel (`.xlsx`)** -- the map as an indented outline worksheet: each topic in the column matching its depth, with a Notes column. For when you want to sort, filter, or count the outline as a spreadsheet.
- **PDF** (via print) -- a fixed-layout document for sending and printing.

A rule of thumb: **JSON to keep it, Markdown to discuss it, PNG/SVG/HTML to show it, the slide**

deck or PowerPoint to present it, Word or PDF to send it, Excel to crunch it.

Copy the outline

Sometimes you don't want a file at all -- you want the text on your clipboard, ready to paste. The **Copy outline** button copies the whole map as a Markdown outline in one click: drop it straight into an email, a chat message, a ticket, or a document. It's the same structure as the Markdown export without the round trip through a saved file -- the fastest way to turn a map into words somewhere else. Reach for it when the map was the *thinking* and some other place is where the writing has to land.

Importing

The same breadth applies going in. MindMap Studio reads:

- **Native** `.json` -- the lossless round trip of the export above.
- **Markdown** outlines -- any `#` /bullet structure becomes a map.
- **OPML** -- from other outliners.
- **FreeMind / Freeplane** `.mm` -- topics, links, folded state, and notes.
- **Mermaid** `mindmap` text -- any node shape; the hierarchy comes from the indentation.
- **XMind** `.xmind` (2020+) -- topics, notes, web links, and labels become tags.
- **MindManager** `.mmap` -- see below.

You can **batch-import** several files at once, which turns a folder of outlines into a library of maps in one step. These bridges cover the common ground between tools; each keeps the topic tree and the fields that map cleanly, and -- like the `.mmap` importer -- quietly leaves behind only the tool-specific extras it can't represent.

The MindManager bridge

If you're arriving from MindManager, the `.mmap` **importer** is the on-ramp. It reads a `.mmap` file and recovers the parts that map cleanly onto MindMap Studio's model: the topic tree and text, notes, stock icons (rendered as emoji markers), hyperlinks, relationships, boundaries, and floating topics. It was built against MindManager's own published schema rather than guessed at, so the common cases come through faithfully.

It is deliberately **one-way and lossy** : MindManager carries project data -- task scheduling, resources, Gantt information -- that MindMap Studio doesn't model, and the importer tells you when it has left something behind rather than pretending the map came across whole. The bridge is for *migrating* a map, not for living in both tools at once.

> **Why lossy is the honest choice.** A converter that silently drops what it can't represent leaves you to discover the gaps later, usually at the worst moment. The importer's warnings are a feature: they tell you exactly what to check.

Where your data lives

Nothing in this chapter sends your map anywhere. Exports are files saved to *your* machine; imports read files *you* choose. Day to day, every map lives in your browser's local database, and the app works fully offline (Chapter 7). Sharing is always an explicit act, never a background one -- which is exactly how a thinking tool should treat the half-formed ideas you trust it with.

Now you try

Take any map and export it twice: once to **JSON** and once to **Markdown** . Open the Markdown in a text editor — there's your outline, ready to paste into a document. Now start a **new** map and import the JSON you saved: it comes back exactly, down to the markers and arrows. You've just proved the thing that matters most about a thinking tool — your work isn't trapped in it. **JSON to keep it, Markdown to share it.** Make that round trip once and you'll trust the app with the ideas you're still figuring out.

Then take the same map to an audience two more ways. Export the **slide deck** (or **PowerPoint** , if the deck has to live there) and open it -- you're presenting: an overview, then one slide per branch, arrow keys to move, nothing installed. Export **Word** and open it in your word processor -- the same map as an editable outline, ready to become minutes or a brief. One map, four jobs -- archived, discussed, presented, and written up -- and not once did your work leave your machine.

Now get it out as a *picture* and as a *paste* . Export **PNG** (or **SVG** for a crisp, scalable version) and drop it into a slide or a chat -- notice the labels, icons, arrows and boundaries all render, because the image carries real text, not a screenshot. Export to **PDF** when it needs to print, or to **Excel** when you want to sort and count the outline as a spreadsheet. Finally, when you just need the words somewhere *now* , click **Copy outline** and paste -- no file, no download, just the map as a Markdown outline wherever your cursor is. Same map, every door out: a file to keep, a picture to show, a sheet to crunch, a paste to drop.

The final chapter is about the most demanding kind of sharing: standing up and walking a room through a map live.

Presenting and workshops

The library, templates, walk-through mode, and running a room

A map is often built to be shown. This chapter covers the features that turn a private canvas into a shared session: managing many maps, starting from the right template, presenting branch by branch, and the offline reliability that lets you do it anywhere.

The library: many maps, one app

MindMap Studio keeps a **library** of named maps in your browser. The map switcher in the toolbar moves between them; **+ New...** starts another; you can **duplicate** the current map (to try a variation without risking the original) and delete ones you're done with. Because each map is independent and locally stored, the library is your filing cabinet -- one map per project, per meeting, per idea -- with cross-map links (Chapter 5) tying related ones together.

Templates: a running start

A blank canvas is the right start for a freeform brainstorm. For everything else, **+ New...** offers templates:

- **Blank** -- a single root, for when the structure is entirely yours.
- **Brainstorm** -- the central idea with the six question-branches from Chapter 2.
- **SWOT** -- Strengths, Weaknesses, Opportunities, Threats, ready to fill in.
- **Project plan** -- Goals, Scope, Milestones, Risks, Team.

A template is just a starting structure you then make your own; its value is saving you the first thirty seconds of "what were the boxes again?" and nudging you toward a complete frame.

Where a template is an *empty* frame, the **Examples** group in the same **+ New…** menu gives you *complete* , worked maps -- a filled launch plan, a sprint retro, a worked SWOT, a trip itinerary, a study map -- to open and adapt rather than build from scratch. They're the fastest way to see what a finished map looks like, and to learn a feature by reading one that uses it.

Walk-through presentation mode

When it's time to present, **Present** enters a focused **walk-through** mode. Instead of showing the whole map at once and asking the room to find the thread, walk-through moves through the map branch by branch, framing each in turn, so attention follows you. Combined with a dark theme (Chapter 4) and a collapsed starting view (Chapter 5), it turns a dense canvas into a guided tour: open the branch under discussion, talk to it, move on.

Presenting from the map itself -- rather than a deck exported from it -- keeps the single source of truth live. A question sends you to a different branch; an idea from the room becomes a node on the spot. The map is the slides *and* the working document.

It works offline, and it installs

MindMap Studio is a **progressive web app** . The first time you load it, it caches its own shell, so afterwards it opens and runs with **no network at all** -- on a plane, in a basement meeting room, anywhere. You can also **install** it to your desktop or home screen, where it launches in its own window like a native app. For a tool you might open to capture a thought the instant you have it, "always available, never loading" matters.

Because it caches itself, it also **updates itself politely** : when a new version ships, the running app shows a small "new version available -- Refresh now" prompt rather than reloading under you mid-thought. Click it when you're ready; an in-flight edit is never lost.

Back up the whole library

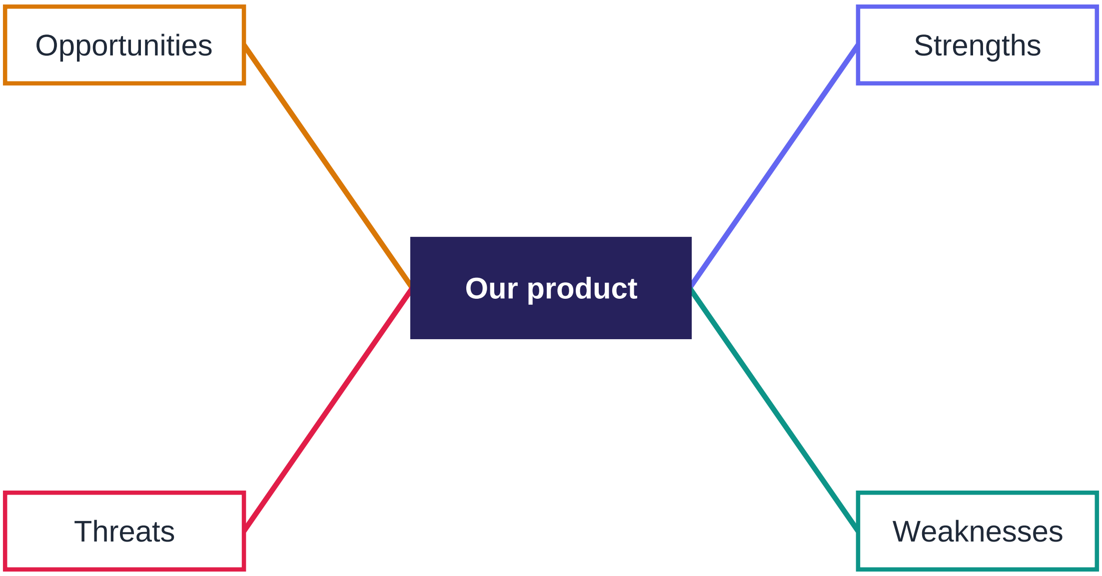
Individual maps export as JSON (Chapter 6). The whole **library** can be backed up and restored in one operation -- a single file with every map in it. Run a backup before a big reorganisation, before switching machines, or just on a schedule you trust. It's the belt to local storage's braces: your maps live on your machine, and a backup means a machine is not a single point of failure.

Running a session: a short playbook

1. **Before:** build or open the map; **collapse all** so it opens at altitude; pick a theme that suits the room.
2. **Open: Fit** , then enter **Present** .
3. **During:** walk branch by branch; capture new ideas as nodes as they come; use `/` to jump when the conversation does.
4. **After:** export to **PNG** or **PDF** for the recap; export **JSON** or run a **library backup** to keep the working map safe.

Now you try

Start a new map from the **SWOT** template. It opens with four branches ready to fill:



A SWOT map: a subject in the centre, four branches to fill in — the SWOT template.

Pick something real -- a product, a project, a decision -- put it in the centre, and spend ten minutes filling each branch. Don't aim for a long list; aim for the *honest* three items per branch. Then look across the four: does an Opportunity answer a Weakness? Does a Threat undercut a Strength? Those cross-links are where a SWOT stops being four lists and starts being analysis -- draw a **relationship** arrow for each one you find. That move, more than the lists themselves, is the thinking.

Then rehearse the room. **Duplicate** the map so you can experiment freely, switch back to the original from the map switcher, and hit **Present** : walk-through frames each branch in turn -- arrow keys to move, the room's attention following yours, one branch at a time. When you're done, run a **library backup** -- one file holding every map, the belt to local storage's braces -- and, if you haven't already, **install** the app to your desktop so it's one click away and runs with no network.

Prefer a running start over a blank SWOT? Open an **Example** instead (**+ New… -> Examples**) -- a filled launch plan, a retro, a trip, a worked SWOT -- and adapt it. Same skills, less blank page; it's also the quickest way to see a feature used in anger.

That's the whole tool, end to end -- from a single node in Chapter 1 to a map you can think with, enrich, navigate, share and present. The appendices that follow are the reference you'll come back to.

Appendix A -- Keyboard reference

The fastest way to build a map is to keep your hands on the keys. This is the full set, grouped by what you're doing.

Building structure

- **Enter** -- add a sibling (a node at the same level as the selected one).
- **Tab** -- add a child (a node one level deeper, under the selected one).
- **Ctrl+Enter** -- insert a line break *inside* the current node's label.
- **Delete** -- remove the selected node and everything beneath it.
- **Arrow keys** -- move the selection between parent, child and siblings.

Editing

- **Double-click** a node, or just start typing on a selected node, to edit its topic.
- **Ctrl+Z** -- undo the last change (model-aware: it reverses the real edit).
- **Ctrl+Y** or **Ctrl+Shift+Z** -- redo.

Finding and moving around

- **/** -- jump straight to Find (searches topics *and* notes).
- **Fit** (toolbar) -- reframe the whole map to the screen.
- **Drag a node** onto another to re-parent it (the whole sub-branch travels).
- **Drag the background** to pan; **scroll / pinch** to zoom.

Panels and views

- **Outline** (toolbar) -- toggle the indented-list view of the map.
- **Notes** (toolbar / node) -- open the Markdown note editor for the selected node.
- **Markers** -- open the marker palette to tag the selected node.
- **Collapse all / Expand all** (toolbar) -- fold or unfold every branch at once.
- **Layout** (toolbar) -- switch between both-sided, right-only and left-only.

> If you only memorise three things: **Tab** goes deeper, **Enter** stays level, and **/** finds anything. Everything else you can reach from the toolbar while you learn.

Appendix B -- Format reference

What goes in, what comes out, and what survives the trip. For the day-to-day guidance on *which* format to reach for, see Chapter 6; this appendix is the detail.

Export formats

- **JSON** (native) -- **lossless** . The complete model: topics, notes, markers, images, boundaries, relationships, styling. The format to use whenever you might re-open the map.
- **Markdown** -- the tree as an outline (`#` root, nested bullets). Round-trippable as structure; canvas-only detail (colour, arrows) is naturally dropped.
- **OPML** -- standard outliner interchange. Structure and topics; not canvas styling.
- **PNG** -- a raster image of the canvas, for slides and chat.
- **SVG** -- a vector image: crisp at any zoom, ideal for high-resolution use.
- **HTML** -- a single self-contained page, openable in any browser.
- **HTML slide deck** -- the walk-through as a standalone, navigable presentation in one self-contained file (an overview slide, then one per branch).
- **PowerPoint (`.pptx`)** -- a real, editable slide deck (an overview slide, then one per branch with its subtree as bullets); a minimal PresentationML package that opens in PowerPoint, Keynote, LibreOffice, and Google Slides.
- **Excel (`.xlsx`)** -- the map as an indented outline worksheet (each topic in the column matching its depth, plus a Notes column); a minimal SpreadsheetML package that opens in Excel, LibreOffice Calc, Numbers, and Google Sheets.
- **Word (`.docx`)** -- the map as an editable outline document (title, indented bulleted topics, notes as italic lines); a minimal Open-XML package that opens in Word, LibreOffice, Pages, and Google Docs.
- **PDF** -- via the browser's print path; a fixed-layout document for sending or printing.

Import formats

- **JSON** (native) -- the lossless round trip of the JSON export.
- **Markdown** -- any `#` /bullet outline becomes a map.
- **OPML** -- outlines from other tools.
- **MindManager** `.mmap` -- one-way, lossy (see below).
- **Batch import** -- select several files at once to create several maps in one step.

What the `.mmap` importer recovers

The MindManager importer was built from MindManager's published XML schema, not guessed at. From a `.mmap` file it recovers:

- the **topic tree** and all topic text;
- **notes** (the plain-text preview MindManager stores);
- **stock icons** , mapped to the closest **emoji** marker;

- **hyperlinks** on nodes;
- **relationships** , as cross-links between nodes;
- **boundaries** drawn over a subtree;
- **floating topics** .

What it deliberately leaves behind

MindManager carries data MindMap Studio doesn't model -- chiefly **task and project information** : schedules, durations, resource assignments, Gantt data. The importer **warns** you about what it skipped rather than dropping it silently, and it does a left-behind check for any topic it couldn't reach. Treat the import as a migration -- a clean start in a new tool -- not as a two-way sync.

A practical note on lossless round trips

Only **JSON in and JSON out** is guaranteed to reproduce a map exactly. If a map matters, keep a JSON export (or a whole-library backup, Chapter 7) as the canonical copy, and treat the other formats as *views* of it -- excellent for sharing, not the thing you archive.

Appendix C -- Further reading

This book is deliberately practical -- enough technique to make the tool pay off, no more. If the *why* of visual thinking has caught your interest, here is where the deeper water is.

On mind mapping itself

The modern popular form of the mind map -- a central image, radiating branches, colour and keywords -- was popularised by **Tony Buzan** , who argued that this "radiant" layout mirrors how association actually works better than a linear list does. His books are the canonical starting point, and worth reading even where you end up disagreeing with the stricter rules he proposed. ("Mind Map" is a term associated with the Buzan Organisation; MindMap Studio uses the phrase descriptively for the diagram type and is an independent project.)

On externalising thought

The broader idea -- that thinking gets better when you get it *out of your head* and onto a surface you can rearrange -- runs through a lot of good writing on note-taking, sketching and "thinking on paper". Look into the literature on **visual note-taking** and **sketchnoting** for the hand-drawn tradition, and the **personal knowledge management** community for the digital one. Mind mapping sits between them: more structured than a sketch, more spatial than an outline.

On structure and decomposition

A mind map is one way to break a big thing into parts. If you find yourself wanting more rigour about *how* to decompose -- mutually exclusive branches, collectively exhaustive coverage, cause-and-effect rather than mere grouping -- the worlds of **issue trees** , **logic trees** and **causal mapping** are the next step out. They trade some of the mind map's freedom for more disciplined structure, and the two skills reinforce each other.

On the tool

MindMap Studio is open source. The application code is licensed under Apache 2.0; this book under Creative Commons BY-NC 4.0. The repository, the issue tracker, the live app, and a dashboard of the project's own metrics are all linked from mindmap-studio.struktureretsundfornuft.dk . If a chapter here describes something the app no longer does exactly that way, the repository is the source of truth -- and a good place to report the drift.

The shortest possible reading list

If you read only one more thing: pick a real problem you're avoiding, open a blank map, put the problem in the centre, and spend twenty minutes branching it. The technique teaches itself faster than any book can, this one included. The reading is for *after* you've felt it work.